

1 chapter11: 近似算法

1.1 导入

1.1.1 组合优化问题

组合优化问题 Π 不是最小化问题就是最大化问题。它由三个部分组成：

1. 一个实例集合 D_Π ;
 2. 对于每个实例 $I \in D_\Pi$, 存在 I 的一个候选解的有限集合 $S_\Pi(I)$;
 3. 实例 $I \in D_\Pi$ 的每个解 $\sigma \in S_\Pi(I)$, 存在一个值 $f_\Pi(\sigma)$, 称为 σ 的解值。
- 如果 Π 是最小化问题, 那么实例 $I \in D_\Pi$ 的最优解 σ^* 满足:

$$\forall \sigma \in S_\Pi(I), f_\Pi(\sigma^*) \leq f_\Pi(\sigma)$$

- 最大化问题的最优解类似定义:

$$\forall \sigma \in S_\Pi(I), f_\Pi(\sigma^*) \geq f_\Pi(\sigma)$$

- 最优解值记为 $OPT(I) = f_\Pi(\sigma^*)$ 。

1.1.2 近似算法

最优化问题 Π 的一个近似算法 A 是一个（多项式时间）算法，给定一个实例 $I \in D_\Pi$ ，算法输出某个解 $\sigma \in S_\Pi(I)$ 。算法 A 得到的解也称为近似解，值 $f_\Pi(\sigma)$ 记为 $A(I)$ 。

数学表达：

- 问题实例： $I \in D_\Pi$
- 可行解集： $S_\Pi(I)$
- 近似解： $\sigma \in S_\Pi(I)$
- 解值表示： $A(I) = f_\Pi(\sigma)$

1.1.3 装箱问题

- **问题定义：** 给定一个大小在 0 和 1 之间的物品集合，将这些物品装到容量为 1 的箱子中，要求使用箱子数目最少（最小化问题）。
- **实例集 D_Π ：** 由所有实例 $I = \{s_1, s_2, \dots, s_n\}$ 组成，其中 $\forall j (1 \leq j \leq n)$, 有 $s_j \in (0, 1]$ 。
- **解集合 $S_\Pi(I)$ ：** 由子集集合 $\sigma = \{B_1, B_2, \dots, B_k\}$ 构成，满足：
 1. σ 是 I 的不相交划分
 2. $\forall j (1 \leq j \leq k), \sum_{s \in B_j} s \leq 1$
- **解值：** $f_\Pi(\sigma) = |\sigma| = k$ （使用的箱子数量）。
- 最优解是符合条件的具有最小子集数的 σ 。
- 令 A 是为每一物品分配一个箱子的（平凡）算法。根据定义（ A 的时间复杂度是多项式级别的）， A 是一个近似算法。但显然， A 不是一个好的近似算法。

1.1.4 差界

- 对于问题的所有实例 I ，希望近似算法 A 满足：

$$|A(I) - OPT(I)| \leq K \quad (K \text{ 为常数})$$

- 定义：若对每个实例 I 都有

$$OPT(I) - K \leq A(I) \leq OPT(I) + K \iff |A(I) - OPT(I)| \leq K$$

则称算法 A 具有有界误差或差界 K

- 这也称为近似算法的绝对性能保证 (absolute performance guarantee)
- 适用于绝对误差敏感的场景 (如资源分配问题中固定容量的偏差)
- 只有很少的 NP 困难问题已知差界

1.1.5 相对性能界

- 定义：设 A 是求解优化问题 Π 的近似算法， I 是 Π 的实例

$$R_A(I) = \max \left(\frac{A(I)}{OPT(I)}, \frac{OPT(I)}{A(I)} \right) \geq 1$$

- 称为解 $A(I)$ 的近似度 (近似比)
- 提供了算法解质量的统一测度，也称相对近似度
- 绝对近似度：

$$R_A = \inf \{r \geq 1 \mid R_A(I) \leq r, \forall I \in D_\Pi\}$$

表示所有实例上的最大近似度

- 渐进近似度：

$$R_A^\infty = \inf \{r \geq 1 \mid \exists N, R_A(I) \leq r, \forall I \in D_\Pi, |I| \geq N\}$$

表示足够大实例上的近似度

1.1.6 近似方案

- 定义：最优化问题的近似方案是算法簇 $\{A_\epsilon\}_{\epsilon>0}$ ，满足

$$R_{A_\epsilon} \leq 1 + \epsilon$$

- 近似方案是近似度收敛于 1 的一系列算法
- 可视为算法 A ，输入实例 I 和误差界 ϵ ，满足

$$R_A(I, \epsilon) \leq 1 + \epsilon$$

- 多项式近似方案 (PAS)：近似方案 $\{A_\epsilon\}_{\epsilon>0}$ 中，每个 A_ϵ 在 $|I|$ 的多项式时间内运行。注意：PAS 的运行时间对 $|I|$ 是多项式，但对 $1/\epsilon$ 可能非多项式

- **完全多项式近似方案 (FPAS)** 是近似方案 $\{A_\epsilon\}_{\epsilon>0}$ ，每个 A_ϵ 在 $|I|$ 和 $1/\epsilon$ 的多项式时间内运行
- **伪多项式时间算法**：在 L 的多项式时间内运行的算法，其中 L 是输入实例的最大数值。示例：0/1 背包问题的动态规划算法（运行时间 $O(nC)$ ）是伪多项式时间算法。注：若算法在 $\log L$ 的多项式时间内运行，则是严格多项式时间算法
- **FPAS 构造方法**：对存在伪多项式时间算法 A 的 NP 难问题：
 1. 对实例 I 的数值进行标度变换和舍入，得到 I'
 2. 将 A 应用于 I' 得到近似解

表 1: 近似算法对比

类型	时间复杂度	解质量	典型问题
PTAS	$O(n^{f(1/\epsilon)})$	$(1 + \epsilon)$ -近似	欧几里得 TSP
FPTAS	$O((1/\epsilon)^k n^c)$	$(1 + \epsilon)$ -近似	0-1 背包
伪多项式算法	$O(N^k n^c)$	可能为精确解	子集和问题

1.1.7 可近似性分类

1. **完全可近似的 (Fully Approximable)**:
 - 定义：对任意小的 $\epsilon > 0$ ，存在 $(1 + \epsilon)$ -近似算法
 - 典型问题：0/1 背包问题
2. **可近似的 (Approximable)**:
 - 定义：存在具有常数比的近似算法（如 2-近似）
 - 典型问题：最小顶点覆盖问题、多机调度问题
3. **不可近似的 (Inapproximable)**:
 - 定义：除非 $P=NP$ ，否则不存在具有常数比的近似算法
 - 典型问题：货郎问题（TSP）

1.2 可近似算法举例

1.2.1 装箱问题

会有四种启发式方法，最先匹配 (FF，分配第一个能装的箱子)，最佳匹配 (BF，分配能装的箱子里面剩余空间最小的)，递减最先匹配 (FFD)，递减最佳匹配 (BFD)。下面我们对 FF 进行近似度分析：

假定：箱子容量为 1 个单位体积，物品大小 $s_i \leq 1$ ($i = 1, 2, \dots, n$)。

定义：

- $FF(I)$ 表示在实例 I 下，使用 FF 算法装入物品时所用的箱子数目

- $OPT(I)$ 表示最优装入时所用的箱子数目

FF 装箱规则说明:

- 根据 FF 装箱规则, 至多有一个非空箱子所装的物品体积 $< \frac{1}{2}$
- 若存在两个这样的箱子 b_i, b_j ($i < j$), 由于它们装的物品体积均 $< \frac{1}{2}$, 根据算法规则, b_j 中的物品应被装入 b_i 而非新箱子

定理 1. 给定若干个物品, 判断它们是否可由两个箱子装下是 NP-完全的。

证明. 可由划分问题 (Partition problem) 规约。划分问题是 Karp 的 21 个 NP-完全问题之一, 该问题表述为: 给定若干个正整数, 问可否将其划分成和相等的两个部分; 换言之, 给定 $c_1, c_2, \dots, c_n \in \mathbb{Z}^+$, 划分问题问是否存在一个 $S \subseteq \{1, 2, \dots, n\}$ 使得

$$\sum_{i \in S} c_i = \sum_{i \notin S} c_i$$

根据一个划分问题实例, 我们可以构造一个装箱问题, 令物品的尺寸为:

$$a_i = \frac{2c_i}{\sum_{j=1}^n c_j}, \quad i = 1, 2, \dots, n$$

同时我们假设这些 $a_i \in (0, 1]$ 。由于 $\sum_{i=1}^n a_i = 2$, 如果这些物品能用两个箱子装下, 那么这两个箱子都正好装满, 这也同时解答了划分问题。这样我们就把一个已知的 NP-完全问题规约为了两个箱子的装箱问题。而显然一个给定的装箱问题的解可在多项式时间内验证, 因此该问题是 NP-完全的。 $\square$

要证明问题 L 是 NP-完全的, 只需证明: 存在性: 已知的某个 NP-完全问题 (如划分问题) 可规约到 L 无关性: 不要求 L 的所有实例都满足某种特性, 只需特定构造的实例能覆盖原问题的解空间

我们之前衡量近似算法采用的是绝对近似比, 定义为最坏情况下算法的求解值与问题的最优值之间的比率。对任意实例 I , 用 $A(I)$ 表示针对实例 I 运行算法 A 后所用箱子数; $OPT(I)$ 表示装完实例 I 中所有物品所用的最少箱子数。若存在常数 $\alpha \geq 1$, 对任意实例 I 有:

$$A(I) \leq \alpha \cdot OPT(I)$$

则算法 A 的近似比至多是 α 。

定理 2. 除非 $P = NP$, 否则装箱问题不存在多项式时间算法具有小于 $\frac{3}{2}$ 的绝对近似比。

证明 (反证法)。假设存在近似比小于 $\frac{3}{2}$ 的多项式时间近似算法 A , 分类讨论:

- 当 $OPT(I) = 2$ 时:

$$A(I) < \frac{3}{2} \times 2 = 3 \implies A(I) = 2 = OPT(I)$$

- 当 $OPT(I) \geq 3$ 时:

$$A(I) \geq 3$$

此时算法 A 可精确判定“是否能用两个箱子装下”的问题, 而该问题是 NP-完全的。若 $P \neq NP$, 则导致矛盾。 $\square$

虽然 $OPT(I) = 2$ 的情况直接揭示了矛盾的核心（算法 A 被迫精确求解 NP-完全问题），但必须验证 $OPT(I) \geq 3$ 时的行为，才能确保反证假设的全局一致性。这是复杂性理论证明中“规约 + 反证”方法的严谨性体现。

定义 1 (渐进近似比). 对任意常数 $\alpha \geq 1$ ，任意实例 I ，若存在常数 k 满足：

$$A(I) \leq \alpha \cdot OPT(I) + k \quad (AAR) \quad (1)$$

则称所有满足上式的 α 的下确界为算法 A 的渐进近似比。其中：

- α 决定当 $OPT(I)$ 充分大时 $A(I)$ 与 $OPT(I)$ 的比值
- k 可以是：
 - 固定常数，或
 - $o(OPT(I))$ (满足 $\lim_{OPT(I) \rightarrow \infty} \frac{k}{OPT(I)} = 0$)
- 当 $k = 0$ 时，渐进近似比退化为绝对近似比

定理 3. 对于任意的实例 I ，有：

$$FF(I) \leq \frac{17}{10}OPT(I) + \frac{4}{5}$$

(常数 ppt 中为 2, BF 同为 $\frac{17}{10}$)

后续再做展开：<https://zhuanlan.zhihu.com/p/685478438>

定理 4. 对于任意的实例 I ，有：

$$FFD(I) \leq \frac{11}{9}OPT(I) + 4$$

(BFD 同为 $\frac{11}{9}$)

1.2.2 ETSP 问题

给定一个平面上 n 个点的集合 S ，找出一个在这些点上的最短长度的游程 t 。这里一个游程是恰好访问每个点一次的一条回路。

ETSP 问题是 TSP 问题的特殊情况，点之间的距离是欧式距离，满足三角不等关系。对于最近邻 (NN) 法：采用贪心法，设 p_1 是任意一个起始点，下一个访问距离 p_1 最近的点，比如 p_2 ，再继续访问距离 p_2 最近的点，直到回到 p_1 。

定理 5. 对于 ESTP 问题所有满足三角不等式的 n 个城市的实例 I ，有：

$$NN(I) \leq \frac{1}{2}(\lceil \log_2 n \rceil + 1)OPT(I).$$

而且，对于每一个充分大的 n ，存在满足三角不等式的 n 个城市的实例 I 使得：

$$NN(I) > \frac{1}{3}(\log_2(n+1) + \frac{4}{3})OPT(I).$$

很明显这里对于 NN 的近似比：

$$R_{NN}(I) = \frac{NN(I)}{OPT(I)} = O(\log n)$$

显然 $R_{NN} = \infty$. 所以 NN 算法不是常数近似比的算法，不能实际应用。

1.2.3 最小顶点覆盖问题

开始时初始化顶点集合 C 为空集，然后重复执行以下操作——任意选取图中的一条边 (u, v) ，将这两个顶点 u 和 v 加入集合 C 中，同时从图中删除这两个顶点及其所有关联边；持续进行这样的操作直到图中所有的边都被删除为止，最终得到的集合 C 就是所求的顶点覆盖。

算法挑选的边构成图的极大匹配 M （即不存在其他匹配 M' 满足 $M \subset M'$ ）；由于覆盖 M 中的边至少需要 $|M|$ 个顶点，因此最小顶点覆盖的大小至少为 $|M|$ ，而 MVC 算法实际得到的覆盖大小为 $2|M|$ ，由此可推导出近似比 $R_{MVC} \leq 2$ 。MVC 是 2-可近似算法。

设 $G = (V, E)$ ，其中顶点集 $V = \{u, v\}$ ，边集 $E = \{(u, v)\}$ 。近似顶点覆盖算法 *APPROX-VERTEX-COVER* 返回的解只能是 $\{u, v\}$ ，而最优解为 $\{u\}$ 或 $\{v\}$ ，此时该算法返回顶点覆盖的规模恰为最优覆盖的两倍。

考虑以下启发式算法来解决顶点覆盖问题：重复选择度数最高的顶点，然后删除其所有关联边。举一个例子来说明这种启发式算法的近似比不为 2。（提示：考虑一个二分图，左边顶点集合中的点的度数相同，右边顶点集合中的点的度数不相同。）

解答：

构造二分图 $G = (V, E)$ ，其中：

$$V = L \cup R$$

$$L = \{l_1, l_2, l_3, l_4, l_5\}$$

$$R = \{r_1, r_2, \dots, r_{11}\}$$

边集 E 包含以下边：

$$\begin{aligned} &\{(l_1, r_1), (l_1, r_3), (l_1, r_4), (l_1, r_6), (l_1, r_7), \\ &(l_2, r_1), (l_2, r_2), (l_2, r_4), (l_2, r_6), (l_2, r_8), \\ &(l_3, r_1), (l_3, r_2), (l_3, r_3), (l_3, r_5), (l_3, r_9), \\ &(l_4, r_1), (l_4, r_2), (l_4, r_3), (l_4, r_5), (l_4, r_{10}), \\ &(l_5, r_1), (l_5, r_2), (l_5, r_3), (l_5, r_4), (l_5, r_{11})\} \end{aligned}$$

1.2.4 多机调度问题

给定有穷作业集 A 和 m 台相同的机器，作业 a 的处理时间为正整数 $t(a)$ ，每一项作业可以在任一机器上处理。如何把作业分配给机器才能使完成所有作业的时间最短？即如何把 A 划分成 m 个不相交的子集 A_i 使得 $\max \{\sum_{a \in A_i} t(a) \mid i = 1, 2, \dots, m\}$ 最小。

贪心法 G-MPS：按输入的顺序分配作业；把每一项作业分配给当前负载最小的机器；如果当前负载最小的机器有 2 台或 2 台以上，则分配给其中的任意一台（比如标号最小的一台）。

定理 6. 对于多机调度问题的每一个有 m 台机器的实例 I ，贪心算法 *G-MPS* 满足：

$$G-MPS(I) \leq \left(2 - \frac{1}{m}\right) OPT(I)$$

其中 $OPT(I)$ 表示最优调度长度。

证明. 首先建立两个基本事实：

1. $\text{OPT}(I) \geq \frac{1}{m} \sum_{a \in A} t(a)$ (m 台机器的最大负载不小于平均负载)
2. $\text{OPT}(I) \geq \max_{a \in A} t(a)$ (最大负载不小于任一作业的处理时间)

设机器 M_j 的负载最大, 即 $\text{G-MPS}(I) = t(M_j)$ 。令 b 是最后被分配给 M_j 的作业。根据贪心算法规则, 在分配 b 时 M_j 的负载最小, 因此有:

$$t(M_j) - t(b) \leq \frac{1}{m} \left(\sum_{a \in A} t(a) - t(b) \right)$$

由此可得:

$$\begin{aligned} \text{G-MPS}(I) &= t(M_j) \\ &\leq \frac{1}{m} \left(\sum_{a \in A} t(a) - t(b) \right) + t(b) \\ &= \frac{1}{m} \sum_{a \in A} t(a) + \left(1 - \frac{1}{m} \right) t(b) \\ &\leq \text{OPT}(I) + \left(1 - \frac{1}{m} \right) \text{OPT}(I) \quad (\text{由事实 2}) \\ &= \left(2 - \frac{1}{m} \right) \text{OPT}(I) \end{aligned}$$

□

给定 m 台机器和 $m(m-1)+1$ 项作业, 其中前 $m(m-1)$ 项作业的处理时间都为 1, 最后一项作业的处理时间为 m 。算法方案是将前 $m(m-1)$ 项作业平均分配给 m 台机器, 每台分配 $m-1$ 项, 最后一项任意分配给一台机器, 此时 $\text{G-MPS}(I) = 2m-1$ 。最优分配方案则是将前 $m(m-1)$ 项作业平均分配给 $m-1$ 台机器, 每台分配 m 项, 最后一项分配给剩下的机器, 此时 $\text{OPT}(I) = m$ 。由此可得 G-MPS 是 2-近似算法。

递降贪心法 DG-MPS : 首先按处理时间从大到小重新排列作业 (处理时间长的作业优先处理), 然后运用 G-MPS 。

时间复杂度分析: DG-MPS 增加排序时间 $O(n \log n)$, 仍然是多项式时间。

定理 7. 对于多机调度问题的每一个有 m 台机器的实例 I , 递降贪心算法 DG-MPS 满足:

$$\text{DG-MPS}(I) \leq \left(\frac{3}{2} - \frac{1}{2m} \right) \text{OPT}(I)$$

其中 $\text{OPT}(I)$ 表示最优调度长度。

证明. 设作业按处理时间从大到小排列为 $a_1, a_2, \dots, a_n$, 考虑负载最大的机器 M_j 和最后分配给 M_j 的作业 a_i , 存在两种情况:

1. **情况 1:** M_j 只有一个作业
 - 此时 $i = 1$, 即 a_i 是处理时间最大的作业
 - 显然 $\text{DG-MPS}(I) = t(a_1) = \text{OPT}(I)$
2. **情况 2:** M_j 有至少两个作业

- 则 $i \geq m + 1$ (因为前 m 个作业已分配给不同机器)
- 由最优解性质可得:

$$\text{OPT}(I) \geq t(a_m) + t(a_{m+1}) \geq 2t(a_{m+1}) \geq 2t(a_i)$$

- 对 DG-MPS 的负载进行分析:

$$\begin{aligned} \text{DG-MPS}(I) &= t(M_j) \\ &\leq \frac{1}{m} \left(\sum_{k=1}^n t(a_k) - t(a_i) \right) + t(a_i) \\ &= \frac{1}{m} \sum_{k=1}^n t(a_k) + \left(1 - \frac{1}{m} \right) t(a_i) \\ &\leq \text{OPT}(I) + \left(1 - \frac{1}{m} \right) \cdot \frac{1}{2} \text{OPT}(I) \\ &= \left(\frac{3}{2} - \frac{1}{2m} \right) \text{OPT}(I) \end{aligned}$$

□

1.3 不可近似问题

定理 8. 对于不要求满足三角不等式的货郎问题, 不存在常数近似比的近似算法, 除非 $P = NP$ 。

证明. 假设存在这样的近似算法 A , 其近似比 $r \leq K$ (K 为常数)。我们将通过构造性证明这与 NP 完全问题 (哈密顿回路问题) 的关系。

构造方法

对任意给定的图 $G = \langle V, E \rangle$, 构造货郎问题实例 I_G 如下:

- 城市集合为 V , 其中 $|V| = n$
- 距离函数定义为:

$$d(u, v) = \begin{cases} 1, & \text{若 } (u, v) \in E \\ Kn, & \text{否则} \end{cases}$$

关键性质

1. 若 G 有哈密顿回路:

$$\text{OPT}(I_G) = n, \quad A(I_G) \leq r \text{OPT}(I_G) \leq Kn$$

2. 若 G 无哈密顿回路:

$$\text{OPT}(I_G) > Kn, \quad A(I_G) \geq \text{OPT}(I_G) > Kn$$

基于算法 A 可设计如下判定算法:

1. 构造 I_G (耗时 $O(n^2)$)

2. 对 I_G 运行 A (多项式时间)
3. 若 $A(I_G) \leq Kn$ 输出 "Yes", 否则输出 "No"

该算法可在多项式时间内判定哈密顿回路问题 (HC)。由于 HC 是 NP 完全的, 故 $P = NP$, 与普遍接受的复杂性假设矛盾。 $\square$

1.4 完全可近似问题

例如对于 0-1 背包问题: 如果用贪心算法 (G-KK), 不断尝试单位价值最大的物品:

定理 9. 对于 0-1 背包问题的任何实例 I , 贪心算法 G 和最优解 $OPT(I)$ 满足:

$$OPT(I) < 2G - KK(I)$$

其中 $KK(I)$ 表示背包的剩余容量。

证明. 设物品按单位重量价值 $\frac{v_i}{w_i}$ 从大到小排列为 $u_1, u_2, \dots, u_n$, u_k 是贪心算法 G 中第一件未装入背包的物品。根据贪心策略的特性:

$$\begin{aligned} OPT(I) &< G - KK(I) + v_k \quad (\text{最优解最多比贪心解多放入 } u_k \text{ 的价值}) \\ &\leq G - KK(I) + v_{\max} \quad (v_k \leq \max_{1 \leq i \leq n} v_i) \\ &\leq 2G - KK(I) \quad (\text{因 } v_{\max} \leq G) \end{aligned}$$

其中关键不等式成立因为:

- 贪心解 G 至少包含一个最大价值物品 (否则可以替换)
- $KK(I)$ 表示贪心解装入后的剩余容量

$\square$

Input: 参数 $k \geq 1$, 实例 I

Output: 近似解 $A_\varepsilon(I)$

```

1  $\varepsilon \leftarrow \frac{1}{k}$ ;
2 将物品按  $\frac{v_i}{w_i}$  降序排列:  $\frac{v_1}{w_1} \geq \dots \geq \frac{v_n}{w_n}$ ;
3 for  $j \leftarrow 0$  to  $k$  do
4   枚举所有  $j$  件物品的子集  $S \subseteq \{1, \dots, n\}$ ;
5   if  $\sum_{i \in S} w_i \leq C$  then
6     用 G-KK 算法装入剩余物品  $\{1, \dots, n\} \setminus S$ ;
7     记录当前解  $A_j(I)$ ;
8   end
9 end
10 输出  $\max\{A_0(I), \dots, A_k(I)\}$ ;

```

Algorithm 1: PTAS 算法

定理 10. 对任意 $k \geq 1$, PTAS 算法满足:

- 时间复杂度: $O(kn^{k+1})$
- 近似比: $R_k = \frac{OPT(I)}{A_\varepsilon(I)} < 1 + \frac{1}{k} = 1 + \varepsilon$

证明. 时间复杂度分析:

- 子集枚举量: $\sum_{j=0}^k \binom{n}{j} = O(kn^k)$
- 每次 G-KK 计算: $O(n)$
- 总时间: $O(kn^{k+1})$

近似比证明: 设最优解 X , 分两种情况:

1. 若 $|X| \leq k$: 算法必得 X (因枚举所有 $\leq k$ 的子集)
2. 若 $|X| > k$:
 - 取 $Y \subset X$ 为前 k 个最大价值物品
 - 设 $Z = X \setminus Y = \{u_{k+1}, \dots, u_r\}$ (保持 $\frac{v_j}{w_j}$ 降序)
 - 设 u_m 为 Z 中第一个未被算法装入的物品
 - 定义 W 为算法在 u_m 前装入的非 $\{u_1, \dots, u_m\}$ 物品

关键推导:

$$\begin{aligned}
 C' &= C - \sum_{j=1}^k w_j - \sum_{j=k+1}^{m-1} w_j \\
 C'' &= C' - \sum_{u_j \in W} w_j < w_m \quad (\text{因 } u_m \text{ 未装入}) \\
 OPT(I) &\leq \sum_{j=1}^k v_j + \sum_{j=k+1}^{m-1} v_j + C' \frac{v_m}{w_m} \\
 &< \sum_{j=1}^k v_j + \sum_{j=k+1}^{m-1} v_j + \sum_{u_j \in W} v_j + v_m \quad (\text{因 } \frac{v_j}{w_j} \geq \frac{v_m}{w_m}) \\
 &< A_\varepsilon(I) + \frac{OPT(I)}{k+1} \quad (\text{因 } (k+1)v_m \leq OPT(I)) \\
 &\Rightarrow R_k < 1 + \frac{1}{k} = 1 + \varepsilon
 \end{aligned}$$

□

- **PTAS (Polynomial-Time Approximation Scheme)**: 对任意小正数 ε , 在输入规模 n 的多项式时间内找到 $(1 + \varepsilon)$ -近似解。运行时间关于 n 的多项式, 但可能关于 $1/\varepsilon$ 指数增长。
- **FPTAS (Fully Polynomial-Time Approximation Scheme)**: 运行时间同时是 n 和 $1/\varepsilon$ 的多项式, 具有实际可行性。

对于存在伪多项式时间算法的 NP 难问题, 构造 FPTAS 的一般方法:

1. 设原问题实例 I 的输入值为 $x_1, \dots, x_k$

2. 伪多项式算法 A 运行时间为 $T(n, \sum x_i)$
3. 进行标度变换: $x'_i = \lfloor \frac{x_i}{f(\varepsilon)} \rfloor$
4. 得到新实例 I'
5. 用算法 A 求解 I' 得解 S'
6. 证明 S' 是 I 的 $(1 + \varepsilon)$ -近似解

Input: 整数 $k > 0$, 实例 $I = (C, \{(s_i, v_i)\}_{i=1}^n)$

Output: 近似解 $A_\varepsilon(I)$

```

1   $K \leftarrow \frac{C}{2(k+1)n}$  ; // 尺度因子
2  ;
3   $C' \leftarrow \lfloor C/K \rfloor$  ;
4  for  $i = 1$  to  $n$  do
5       $s'_i \leftarrow \lfloor s_i/K \rfloor$  ;
6       $v'_i \leftarrow \lfloor v_i/K \rfloor$  ;
7  end
8  创建新实例  $I' = (C', \{(s'_i, v'_i)\}_{i=1}^n)$ ;
9   $S \leftarrow \text{KNAPSACK}(I')$  ; // 应用伪多项式算法
10 ;
11 return  $S$ 

```

Algorithm 2: FPTAS for 0-1 Knapsack

性质:

- 构成算法簇: 对每个 k , $\varepsilon = 1/k$, 记为 A_ε
- 时间复杂度: $\Theta(n^2/\varepsilon)$

定理 11. 对每个整数 $k > 1$, $\varepsilon = 1/k$ 和任意实例 I :

$$OPT(I) < (1 + \varepsilon)A_\varepsilon(I)$$

时间复杂度为 $\Theta(n^2/\varepsilon)$ 。

证明. 时间复杂度分析:

$$\begin{aligned} T(n) &= \Theta(nC'/K) \\ &= \Theta\left(n \cdot \frac{C}{K} \cdot \frac{1}{K}\right) \\ &= \Theta\left(n \cdot \frac{2(k+1)n}{K}\right) \\ &= \Theta(kn^2) = \Theta(n^2/\varepsilon) \end{aligned}$$

近似比证明：设 I' 为标度变换后的实例：

- $\text{OPT}(I') \geq \sum_{i \in S^*} \lfloor v_i/K \rfloor$ (S^* 是最优解)

- 有不等式链：

$$\text{OPT}(I) - K \cdot \text{OPT}(I') \leq Kn \quad (\text{最优解至多 } n \text{ 项})$$

$$\text{OPT}(I) - A_\varepsilon(I) \leq Kn \quad (\text{近似解定义})$$

$$\begin{aligned} A_\varepsilon(I) &\geq \text{OPT}(I) - Kn \\ &= \text{OPT}(I) - \frac{C}{2(k+1)} \end{aligned}$$

- 假设 $\text{OPT}(I) \geq C/2$ (否则易得最优解), 则:

$$\begin{aligned} \frac{A_\varepsilon(I)}{\text{OPT}(I)} &\geq 1 - \frac{1}{2(k+1)\text{OPT}(I)/C} \\ &> 1 - \frac{1}{k+1} = \frac{k}{k+1} \\ \Rightarrow R_{A_\varepsilon} &< \frac{k+1}{k} = 1 + \frac{1}{k} = 1 + \varepsilon \end{aligned}$$

□

1.5 课后作业

1.5.1 独立集问题

给出一个独立集问题的近似算法，即找到最大的顶点子集，子集中顶点互不相连。并请证明或否定该近似算法的近似度是有界的。

使用贪心算法：

1. 初始化独立集 $S = \emptyset$ 。
2. 从图中选择一个度数最小的顶点 v ，将其加入 S 。
3. 将 v 及其所有邻居从图中删除。
4. 重复步骤 2-3，直到图中没有剩余顶点。

每次选择一个顶点加入 S 时，最多会“排除” $\Delta + 1$ 个顶点（包括该顶点及其邻居）。因此，贪心算法得到的独立集大小至少为 $\frac{\text{OPT}}{\Delta+1}$ ，其中 OPT 是最优解的大小。

1.5.2 最大子集和问题

最大子集和问题：给定含有 n 个正整数的集合 $S = \{s_1, s_2, \dots, s_n\}$ 和一个正整数 C ，要求找出 S 的一个子集，使得它们的和最大且不超过 C 。请给出求解该问题的完全多项式近似方案。要求分析算法时间复杂度和近似比。

运用动态规划思想：

1. 设定近似参数 $\varepsilon \in (0, 1)$ ，计算缩放因子 $k = \frac{\varepsilon \cdot \max(S)}{n}$ 。
2. 对集合 S 中每个元素 s_i ，计算缩放后的值 $s'_i = \lfloor \frac{s_i}{k} \rfloor$ 。
3. 使用动态规划求解缩放后的问题：

- 定义 $dp[i][j]$ 表示前 i 个元素能否组成和 j
- 初始化 $dp[0][0] = \text{True}$
- 状态转移: $dp[i][j] = dp[i-1][j] \vee dp[i-1][j-s'_i]$

4. 在满足 $\sum_{i \in T} s'_i \leq \lfloor \frac{C}{k} \rfloor \triangleq J_{\max}$ 的所有子集 T 中, 利用 $dp[n][..]$ 来找到使 $\sum_{i \in T} s_i$ 最大的解。

而对于该算法的时间复杂度与近似比:

• 参数缩放阶段:

- 计算最大值 $\max(S)$: $O(n)$
- 计算缩放因子 $k = \frac{\varepsilon \cdot \max(S)}{n}$: $O(1)$
- 缩放所有元素 $s'_i = \lfloor \frac{s_i}{k} \rfloor$: $O(n)$

• 动态规划阶段:

- 最大缩放和计算:

$$J_{\max} = \left\lfloor \frac{C}{k} \right\rfloor = O\left(\frac{nC}{\varepsilon \max(S)}\right)$$

- 状态转移次数:

$$O(n \cdot J_{\max}) = O\left(n \cdot \frac{nC}{\varepsilon \max(S)}\right)$$

• 总时间复杂度:

$$T(n, \varepsilon) = O\left(\frac{n^2 C}{\varepsilon \max(S)}\right)$$

注意到 $\max(S) \leq C$, 因此:

$$T(n, \varepsilon) \leq O\left(\frac{n^2}{\varepsilon}\right)$$

显然, 这是一个完全多项式近似算法。

• 缩放误差分析:

$$\begin{aligned} \forall s_i : k \cdot s'_i \leq s_i \leq k(s'_i + 1) \\ \sum_{i \in T} s_i - k \sum_{i \in T} s'_i \leq nk = \varepsilon \max(S) \end{aligned}$$

• 近似比推导:

$$ALG \geq k \cdot OPT' \geq OPT - nk = OPT - \varepsilon \max(S)$$

$$\begin{aligned} R_{ALG} &= \max\left(\frac{ALG}{OPT}, \frac{OPT}{ALG}\right) \\ &= \frac{OPT}{ALG} \\ &\leq \frac{OPT}{OPT - \varepsilon \max(S)} \\ &= \frac{1}{1 - \frac{\varepsilon \max(S)}{OPT}} \end{aligned}$$

若 $\max(S) \leq OPT$, 则易知 $R_{ALG} \leq 1 + \varepsilon$, 若 $\max(S) > OPT$, 则对该算法进行修正, 将所有满足 $s_i > C$ 的 s_i 均删除, 再次进行计算, 即可转化为上一种情况。

其中 ALG 为该算法得到的解, OPT 为最优解, OPT' 为缩放后问题的最优解

1.5.3 无向图最大割集问题

设无向图 $G = (V, E)$, $V_1 \cup V_2 = V, V_1 \cap V_2 = \emptyset$ 。若边 $(u, v) \in E$ 且 $u \in V_1$, 则称 (u, v) 为割边; 否则称为非割边。求最大割集问题: 任给无向图 $G = (V, E)$, 求 G 的边数最多的割集。

求最大割集的局部改进算法 MCUT 思想如下:

初始令 $V_1 = V, V_2 = \emptyset$ 。重复以下操作:

1. 若存在顶点 u , 在 u 关联的边中非割边数 $>$ 割边数

- 若 $u \in V_1$, 则将 u 移至 V_2
- 若 $u \in V_2$, 则将 u 移至 V_1

2. 直到不存在这样的顶点时停止

最终得到的 (V_1, V_2) 即为解。对任意实例满足:

$$OPT(I) \leq 2 \cdot MCUT(I)$$

其中 $OPT(I)$ 表示最优解的边数。

设 C 为算法得到的割边数 $MCUT(I)$, 设顶点 u 的割边与非割边数分别是 $c(u)$ 和 $nc(u)$, 那么:

每条割边连接 V_1 和 V_2 , 因此 $\sum_{u \in V} c(u) = 2C$ 。因为 $c(u) \geq nc(u)$, 所以对于每个顶点 u , 其度数 $d(u) = c(u) + nc(u) \leq 2 \times c(u)$, 那么 $c(u) \geq \frac{d(u)}{2}$, 所以 $2C = \sum_{u \in V} c(u) \geq \frac{1}{2} \sum_{u \in V} d(u) = |E|$, 所以 $OPT(I) \leq |E| \leq 2C = 2 \cdot MCUT(I)$

1.5.4 平面 TSP 问题的最小权匹配

对 TSP 问题所有满足三角不等关系的实例 I , 已知有最小权匹配 (MM) 近似算法求解, 且 $MM(I) < \frac{3}{2}OPT(I)$, 请给出具有该近似比的紧实例。

这个题肯定需要让顶点数 n 趋于无穷, 使得 $length(T) \rightarrow OPT(I), length(M) \rightarrow \frac{1}{2}OPT$ 即可, 这里我们必须约定我们找到的最小生成树满足某些特殊的性质, 使得 n 个点中度数为奇数的点的数量是接近 n 的, 这样就可以使得 $length(M) \rightarrow \frac{1}{2}OPT$ 。而且依靠这个最小生成树得到的欧拉图, 必须是能够变成哈密尔顿回路的, 即不存在割点。

按照以上原则进行构造即可。

- **PTAS (Polynomial-Time Approximation Scheme)**: 对任意小正数 ε , 在输入规模 n 的多项式时间内找到 $(1 + \varepsilon)$ -近似解。运行时间关于 n 的多项式, 但可能关于 $1/\varepsilon$ 指数增长。
- **FPAS/FPTAS (Fully Polynomial-Time Approximation Scheme)**: 运行时间同时是 n 和 $1/\varepsilon$ 的多项式, 具有实际可行性。

对于存在伪多项式时间算法的 NP 难问题, 构造 FPTAS 的一般方法:

1. 设原问题实例 I 的输入值为 $x_1, \dots, x_k$
2. 伪多项式算法 A 运行时间为 $T(n, \sum x_i)$
3. 进行标度变换: $x'_i = \lfloor \frac{x_i}{f(\varepsilon)} \rfloor$
4. 得到新实例 I'

5. 用算法 A 求解 I' 得解 S'
6. 证明 S' 是 I 的 $(1 + \varepsilon)$ -近似解

Input: 整数 $k > 0$, 实例 $I = (C, \{(s_i, v_i)\}_{i=1}^n)$

Output: 近似解 $A_\varepsilon(I)$

```

1  $K \leftarrow \frac{C}{2(k+1)n}$ ; // 尺度因子
2 ;
3  $C' \leftarrow \lfloor C/K \rfloor$ ;
4 for  $i = 1$  to  $n$  do
5    $s'_i \leftarrow \lfloor s_i/K \rfloor$ ;
6    $v'_i \leftarrow \lfloor v_i/K \rfloor$ ;
7 end
8 创建新实例  $I' = (C', \{(s'_i, v'_i)\}_{i=1}^n)$ ;
9  $S \leftarrow \text{KNAPSACK}(I')$ ; // 应用伪多项式算法
10 ;
11 return  $S$ 
```

Algorithm 3: FPTAS for 0-1 Knapsack

性质:

- 构成算法簇: 对每个 k , $\varepsilon = 1/k$, 记为 A_ε
- 时间复杂度: $\Theta(n^2/\varepsilon)$

定理 12. 对每个整数 $k > 1$, $\varepsilon = 1/k$ 和任意实例 I :

$$\text{OPT}(I) < (1 + \varepsilon)A_\varepsilon(I)$$

时间复杂度为 $\Theta(n^2/\varepsilon)$ 。

证明. 时间复杂度分析:

$$\begin{aligned}
T(n) &= \Theta(nC'/K) \\
&= \Theta\left(n \cdot \frac{C}{K} \cdot \frac{1}{K}\right) \\
&= \Theta\left(n \cdot \frac{2(k+1)n}{K}\right) \\
&= \Theta(kn^2) = \Theta(n^2/\varepsilon)
\end{aligned}$$

近似比证明: 设 I' 为标度变换后的实例:

- $\text{OPT}(I') \geq \sum_{i \in S^*} \lfloor v_i/K \rfloor$ (S^* 是最优解)
- 有不等式链:

$$\text{OPT}(I) - K \cdot \text{OPT}(I') \leq Kn \quad (\text{最优解至多 } n \text{ 项})$$

$$\text{OPT}(I) - A_\varepsilon(I) \leq Kn \quad (\text{近似解定义})$$

$$\begin{aligned}
A_\varepsilon(I) &\geq \text{OPT}(I) - Kn \\
&= \text{OPT}(I) - \frac{C}{2(k+1)}
\end{aligned}$$

- 假设 $\text{OPT}(I) \geq C/2$ (否则易得最优解), 则:

$$\begin{aligned}\frac{A_\epsilon(I)}{\text{OPT}(I)} &\geq 1 - \frac{1}{2(k+1)\text{OPT}(I)/C} \\ &> 1 - \frac{1}{k+1} = \frac{k}{k+1} \\ \Rightarrow R_{A_\epsilon} &< \frac{k+1}{k} = 1 + \frac{1}{k} = 1 + \epsilon\end{aligned}$$

□

1.5.5 最大团问题

给定近似参数 ϵ , 假设存在一个近似比为常数 ρ 的算法 A 用于求解最大团问题。对于图 $G = (V, E)$, 设其最大团大小为 m , 我们通过以下步骤构造新的近似算法:

Input: 图 $G = (V, E)$, 近似参数 ϵ
Output: 大小为 $n^{1/k}$ 的团 C

- 1 计算 $k = \lceil 1/\log_c(1 + \epsilon) \rceil$; // c 为算法 A 的近似比
- 2 构造图 $G^{(k)}$; // 通过 G 的 k -幂运算
- 3 使用算法 A 在 $G^{(k)}$ 中找到大小为 n 的团
- 4 将 $G^{(k)}$ 的团映射回 G , 得到团 C
- 5 **return** C

Algorithm 4: 完全多项式时间近似模式 A'

$$\begin{aligned}\frac{m}{n^{1/k}} &\leq c^{1/k} \\ \text{令 } k &= \frac{1}{\log_c(1 + \epsilon)} \\ \Rightarrow \frac{m}{n^{1/k}} &\leq 1 + \epsilon \\ \Rightarrow n^{1/k} &\geq \frac{m}{1 + \epsilon}\end{aligned}$$

$$\begin{aligned}k &= \frac{1}{\log_c(1 + \epsilon)} = \frac{\ln c}{\ln(1 + \epsilon)} \\ &\geq \frac{\ln c}{\epsilon} \quad (\text{因为 } \ln(1 + \epsilon) \leq \epsilon)\end{aligned}$$

因此算法 A' 满足:

- 近似比: $1 + \epsilon$
- 时间复杂度: 关于 $|V|$ 和 $1/\epsilon$ 的多项式

上述构造证明了最大团问题存在完全多项式时间近似模式 (FPTAS), 其中:

$$\text{输出团大小} \geq \frac{m}{1 + \epsilon}$$

且运行时间关于输入规模和 $1/\epsilon$ 均为多项式级别。

1.5.6 证明 TSP 是不可近似问题

反证法. 假设存在常数 $\rho \geq 1$, 使得一般旅行商问题 (TSP) 存在多项式时间 ρ -近似算法 A 。我们将通过以下步骤导出矛盾:

步骤 1: 问题转化 给定哈密顿环问题实例 $G = (V, E)$, 构造对应的 TSP 实例:

- 完全图 $G' = (V, E')$, 其中 $E' = \{(u, v) : u, v \in V \text{ 且 } u \neq v\}$
- 代价函数定义为:

$$c(u, v) = \begin{cases} 1 & \text{若 } (u, v) \in E \\ \rho|V| + 1 & \text{否则} \end{cases}$$

步骤 2: 代价分析

- **情况 1:** 若 G 有哈密顿环 H , 则对应 TSP 回路代价:

$$C_{\text{opt}} = \sum_{(u,v) \in H} c(u, v) = |V|$$

- **情况 2:** 若 G 无哈密顿环, 则任何 TSP 回路至少包含一条非 E 中的边, 其最小代价为:

$$C_{\min} \geq (\rho|V| + 1) + (|V| - 1) > \rho|V|$$

步骤 3: 算法行为 对构造的 TSP 实例 $\langle G', c \rangle$ 运行算法 A :

- 若 G 有哈密顿环, A 输出回路代价 $C_A \leq \rho C_{\text{opt}} = \rho|V|$
- 若 G 无哈密顿环, A 输出回路代价 $C_A > \rho|V|$

步骤 4: 矛盾导出 通过比较 C_A 与 $\rho|V|$ 可判定 G 是否存在哈密顿环。这意味着:

$$\text{哈密顿环问题} \in P$$

根据定理 34.13 (哈密顿环的 NP 完全性) 和定理 34.4 ($P=NP$ 的充分条件), 将导致 $P = NP$, 与前提 $P \neq NP$ 矛盾。

□

Input: 哈密顿环问题实例 $G = (V, E)$

Output: 判定 G 是否存在哈密顿环

- 1 构造完全图 $G' = (V, E')$, 其中 $E' = \{(u, v) : u, v \in V \text{ 且 } u \neq v\}$;
- 2 定义代价函数:

$$c(u, v) = \begin{cases} 1 & \text{若 } (u, v) \in E \\ \rho|V| + 1 & \text{否则} \end{cases}$$

- 3 运行 ρ -近似算法 A 于 $\langle G', c \rangle$;
- 4 **if** A 输出回路代价 $\leq \rho|V|$ **then**
- 5 **return** 存在哈密顿环;
- 6 **else**
- 7 **return** 不存在哈密顿环;
- 8 **end**

Algorithm 5: TSP 不可近似性的归约过程

理论意义

该证明揭示了:

- TSP 的不可近似性直接源于其与 NP 完全问题的等价关系
- 近似算法存在性将导致计算复杂性层级坍塌 ($P = NP$)
- 对任意 $\rho \geq 1$ 的推广通过代价函数的伸缩性实现

1.5.7 瓶颈旅行商问题

瓶颈旅行商问题 (bottleneck traveling-salesman problem) 要求找到一条回路, 满足该回路中代价最大的边的代价是所有回路中代价最大的边的代价中最小的。假设代价函数满足三角形不等式, 证明: 该问题有一个近似比为 3 的多项式时间近似算法。

给定无向完全图 $G = (V, E)$, 根据思考题 21-4 的结论, 可以在关于 $|V|$ 和 $|E|$ 的线性时间内求出一棵瓶颈生成树 T 。因为 G 是完全图, 根据练习题 34.2-11 的结论, 一定能由 T 在关于 $|V|$ 和 $|E|$ 的多项式时间内构造出一个哈密顿环 H , 且 H 上任意两个连续顶点 u 和 v 间在 T 上的距离不超过 3 条边。设 B_T 表示瓶颈生成树中代价最高的边。

近似比证明. 根据三角形不等式 (如图 35.2-4 所示):

$$\begin{aligned} c(v_1^{(1)}, v_2^{(2)}) &\leq c(v_1^{(1)}, v_2^{(1)}) + c(v_2^{(1)}, v_2^{(2)}) \\ &\leq c(v_1^{(1)}, u) + c(u, v_2^{(1)}) + c(v_2^{(1)}, v_2^{(2)}) \\ &\leq 3c(B_T) \end{aligned}$$

因此对任意边 $(u, v) \in H$, 有 $c(u, v) \leq 3c(B_T)$ 。

设 H 中代价最大的边为 B_H , 则:

$$c(B_H) \leq 3c(B_T) \quad (2)$$

设最优瓶颈回路 H^* 中代价最大的边为 B_H^* ，将其移除后得到生成树。根据瓶颈生成树的定义：

$$c(B_T) \leq c(B_H^*) \quad (3)$$

联立 (1)(2) 式可得：

$$c(B_H) \leq 3c(B_H^*)$$

□

1.5.8 最优回路不会自我交叉

假设旅行商问题的一个实例的所有顶点是一组平面上的点，并且代价 $c(u, v)$ 是点 u 和 v 之间的欧氏距离。证明：一条最优回路不会自我交叉。

采用反证法，假设存在一条最优回路存在一个自我交叉，如图 35.2-5 所示，设 (v_1, v_2) 和 (v_3, v_4) 交叉位于 x ，有

$$c(v_1, v_2) + c(v_3, v_4) = c(v_1, x) + c(v_2, x) + c(v_3, x) + c(v_4, x)$$

因为所有顶点都位于欧式平面上且代价 $c(u, v)$ 是点 u 和 v 之间的欧氏距离。所以 c 满足三角形不等式，有 $c(v_1, x) + c(v_3, x) > c(v_1, v_3)$ 且 $c(v_2, x) + c(v_4, x) > c(v_2, v_4)$ 。代入前面的等式，有 $c(v_1, v_2) + c(v_3, v_4) > c(v_1, v_3) + c(v_2, v_4)$ ，用边 (v_1, v_3) 和 (v_2, v_4) 替换原回路中的 (v_1, v_2) 和 (v_3, v_4) 得到一个代价更小的回路，与原回路是最优回路相矛盾。因此，一条最优回路不会自我交叉。证明完毕。

1.5.9 TSP 不可近似性

证明对于任意常数 $c \geq 0$ ，一般旅行商问题不存在具有近似比为 $|V|^c$ 的多项式时间的近似算法。

证明：采用反证法。假设对于某个常数 $c \geq 0$ ，一般旅行商问题存在近似比为 $|V|^c$ 的多项式时间算法 A 。我们将说明如何使用 A 来在多项式时间内求解哈密顿环问题的一个实例。根据定理 34.13，哈密顿环问题是 NP 完全的，根据定理 34.4，若哈密顿环问题存在一个多项式时间算法，则 $P = NP$ 。

设 $G = (V, E)$ 是哈密顿环问题的一个实例，使用假想的近似算法 A 来检测 G 是否包含一个哈密顿环。按照如下方法将 G 转化为旅行商问题的一个实例，设 $G' = (V, E')$ 为关于 V 的完全图，即 $E' = \{(u, v) : u, v \in V \text{ and } u \neq v\}$ 。其代价函数为：

$$c(u, v) = \begin{cases} 1 & \text{if } (u, v) \in E, \\ |V|^{c+1} + 1 & \text{otherwise.} \end{cases}$$

可以在关于 $|V|$ 和 $|E|$ 的多项式时间内构造出 G' 和 c 。

现在考虑旅行商问题的一个实例 $\langle G', c \rangle$ 。若 G 中存在一个哈密顿环 H ，则 c 赋予 H 中每条边的代价为 1， H 的代价为 $|V|$ ， $\langle G', c \rangle$ 中有一条回路代价为 $|V|$ 。若 G 中不存在一个哈密顿环，则 G' 中的回路必须包含不属于 E 的边，该回路的代价至少为 $(|V|^{c+1} + 1) + (|V| - 1) = |V|^{c+1} + |V| = |V|^c(|V| + 1) > |V|^c \cdot |V|$ 。即该回路不是 G 中的一个哈密顿环的因子超过 $|V|^c$ 。

假设我们用近似算法 A 来求解旅行商问题 $\langle G', c \rangle$ 。因为 A 保证能够返回的回路代价不超过最优回路代价的 $|V|^c$ 倍。若 G 中包含一个哈密顿环，则 A 返回一个满足上述要求的回路。若 G 中不包含一个哈密顿环，则 A 返回一个代价大于 $|V|^{c+1}$ 的回路。因此，可以使用 A 在多项式时间内求解哈密顿环问题。